

PYTHON

PROYECTO

Lorena Binet Díaz



LORENA BINET DÍAZ

RESPONSABLE DEL DESARROLLO DE LA APLICACIÓN

Fecha: 10-02-2026



ÍNDICE DE CONTENIDOS

1. INTRODUCCIÓN Y DESCRIPCIÓN GENERAL	Pág. 4-5
• 1.1. Concepto del Proyecto: Web de Torneos de Videojuegos	
• 1.2. Justificación de la Estética Visual y Experiencia de Usuario	
2. OBJETIVOS Y ALCANCE DEL PROYECTO	Pág. 6-8
• 2.1. Objetivos Generales y Específicos	
• 2.2. Alcance Funcional: Módulos del Sistema	
• 2.3. Limitaciones y Exclusiones	
3. STACK TECNOLÓGICO Y ALTERNATIVAS	Pág. 3
• 3.1. Tecnologías de Backend	
• 3.2. Tecnologías de Frontend	
• 3.3. Herramientas de Desarrollo y Despliegue	
• 3.4. Evaluación de Alternativas Técnicas	
4. MODELO DE DATOS Y ESQUEMA	Pág. 4
• 4.1. Arquitectura de la Base de Datos	
• 4.2. Esquema de la Base de Datos	
5. REQUISITOS DE LA APLICACIÓN	Pág. 5
• 5.1. Requisitos Funcionales	
• 5.2. Requisitos No Funcionales	
6. MANUAL DE INSTALACIÓN	Pág. 6
7. CONCLUSIONES	Pág. 7
8. EVOLUTIVOS DEL PROYECTO	Pág. 8



DESCRIPCIÓN GENERAL DEL PROYECTO

1.1. Concepto del Proyecto: Torneos All Stars

La idea de este proyecto, al que he llamado **Torneos All Stars**, es crear una plataforma web para organizar torneos de videojuegos, que fuera inmersiva, que se viera como parte de un videojuego. Esta web tiene acceso a diferentes videojuegos para que el usuario pueda elegir libremente dentro de la variedad que la web ofrece.

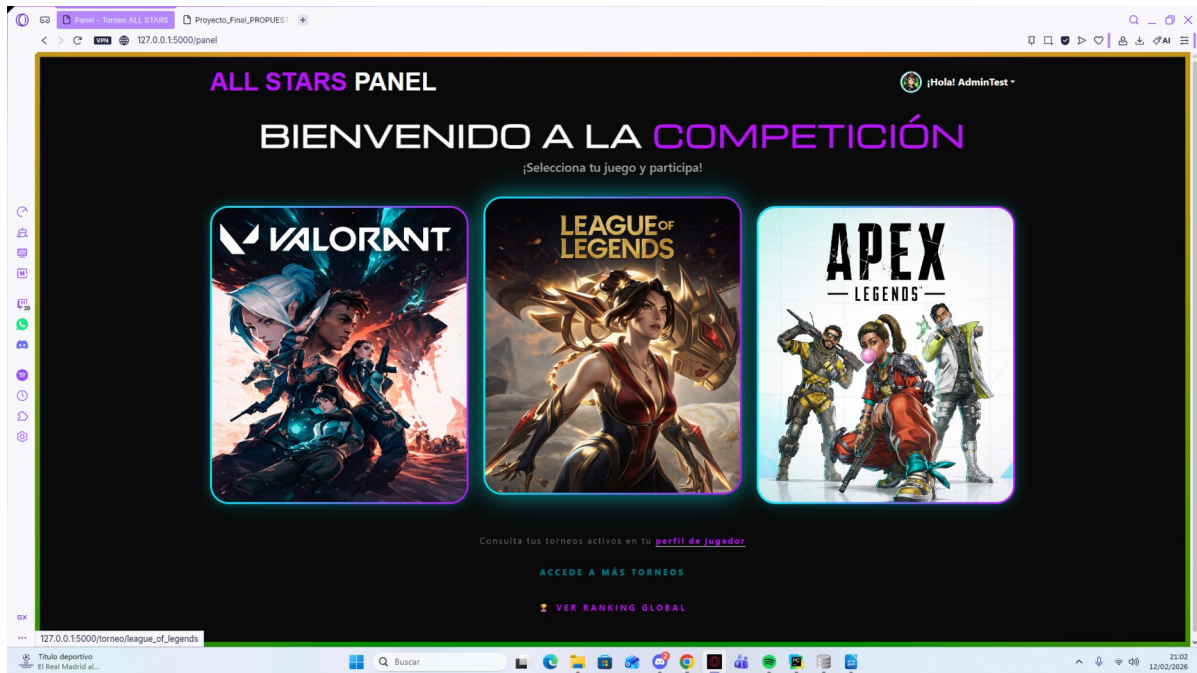
Los usuarios pueden entrar, ver qué torneos hay disponibles e inscribirse. Con este proyecto buscaba que el usuario no sintiera que está rellenando un formulario, sino que está entrando a un sistema táctico de videojuegos.

1.2. Justificación de la Estética Visual y Experiencia de Usuario

Buscaba que la estética visual de la web fuera futurista, simulando el mundo “gaming”, por lo que me inspiré en la estética de **Cyberpunk**.

Para conseguirlo, usé referencias de *Neumorphism.io* para entender cómo funcionan las sombras duales. Además encontré una tendencia que se llama *Glassmorphism*, que usa transparencias.

También, buscaba que la web fuera fácil de usar, aplicando una jerarquía por colores, utilizando diferentes estilos de fuentes para que sea legible, una organización intuitiva, además de efectos visuales cuando el usuario pasa el ratón por los diferentes elementos interactivos.





OBJETIVOS Y ALCANCE DEL PROYECTO

2.1. Objetivos Generales del Proyecto

El objetivo de este proyecto no era solo hacer una página web que se viera bien, sino conseguir que **fuera funcional y útil**. He dividido mis metas en dos partes:

Objetivos generales:

- **Desarrollar una aplicación web completa:** Quería entender cómo conectar el diseño que ve el usuario (Frontend) con la lógica y la base de datos que hay detrás (Backend) usando **Python y Flask**.
- **Crear una experiencia inmersiva:** Quería que la web tuviera una estética de videojuegos.

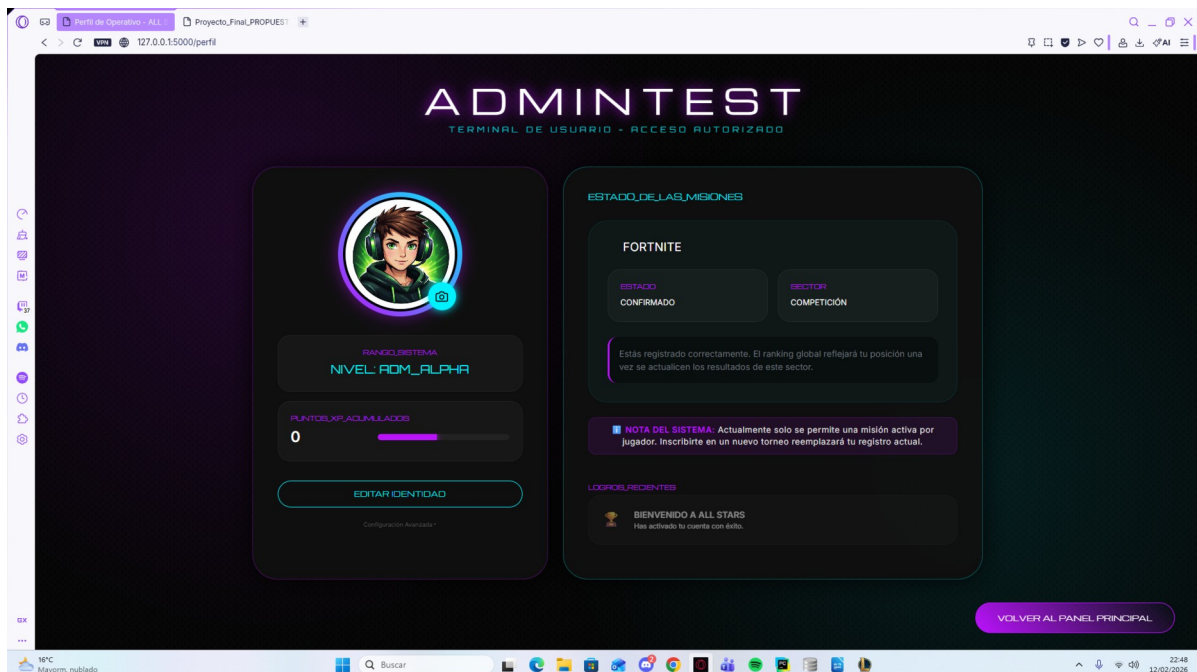
Objetivos específicos:

- **Gestión de datos real:** Conseguir que el administrador pueda agregar los puntos que los participantes de los torneos ganan a lo largo de la duración de dicho torneo, que también pudiera editar el rol y el nivel del usuario, además de eliminarlos, y que esos cambios se guarden en la base de datos **SQLite**.
- **Aprender JavaScript (Fetch API):** Cuando elegí la propuesta de la web de torneos de videojuegos, quise implementar un sistema de puntos que pudiera gestionar el administrador, pero no sabía muy bien cómo hacer que el usuario viera sus puntos al instante sin que tuviera que recargar la página, así que busqué información y aprendí cómo hacerlo a grandes rasgos.
- **Visualización de datos:** Implementar una gráfica dinámica (usando **Chart.js**) que muestre al administrador qué juegos tienen más éxito entre los participantes.

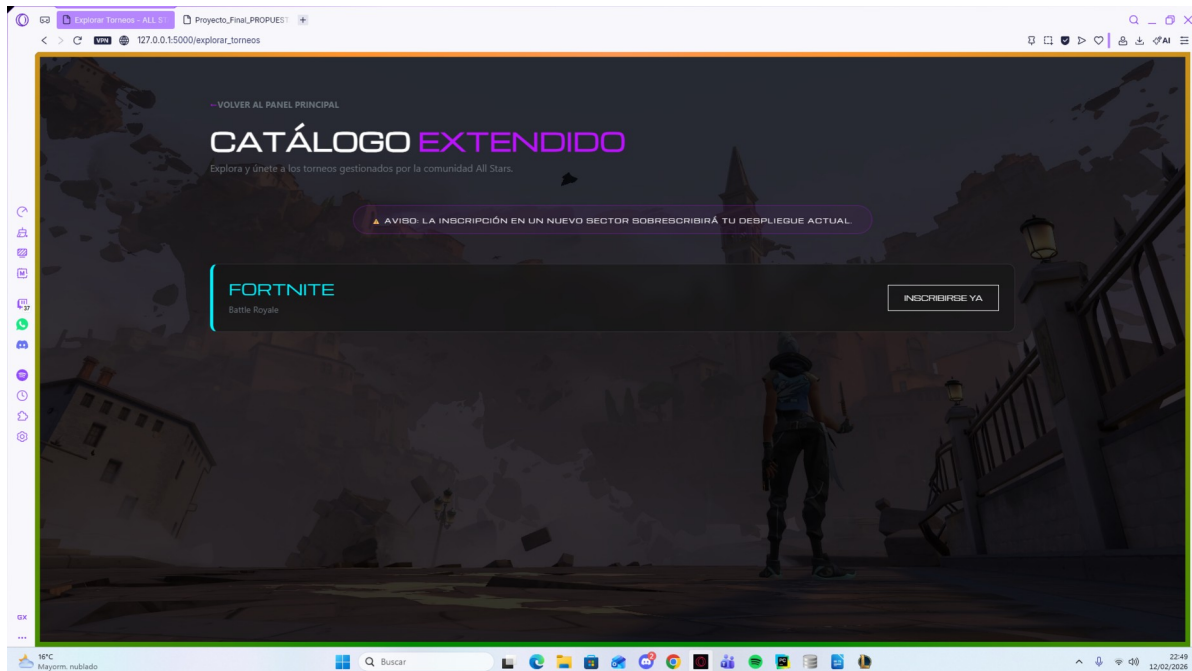
2.2. Alcance Funcional: Módulos del Sistema

Para que la web sea organizada, la he dividido en tres módulos principales que cubren todo lo que el usuario y el administrador necesitan:

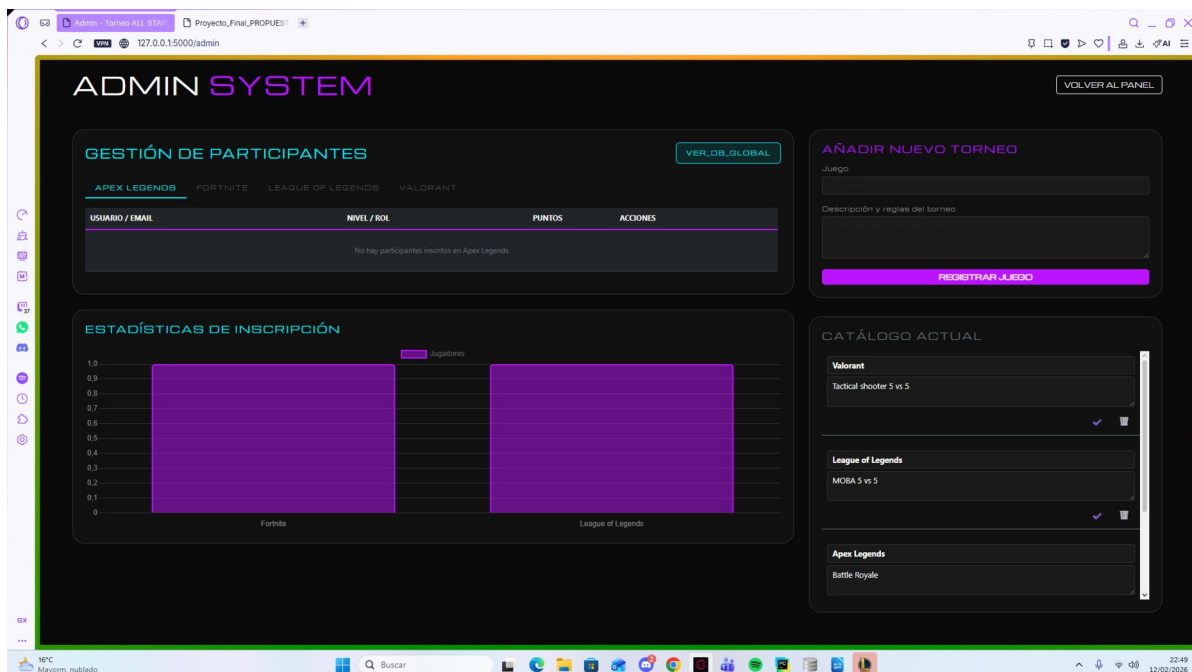
- **Módulo de Identidad (Perfil):** Es el espacio donde cada jugador gestiona su información. Permite ver los puntos que ha ganado en el torneo en el que está participando, su nivel y la misión (juego) que tiene activa en ese momento.



- **Módulo de Catálogo de Juegos:** Una sección dinámica donde se listan todos los torneos disponibles.



- **Módulo de Control Administrativo:** Incluye la gestión de participantes por pestañas, la capacidad de añadir o eliminar torneos del catálogo y un buscador global de usuarios para ver quién está registrado en la base de datos.



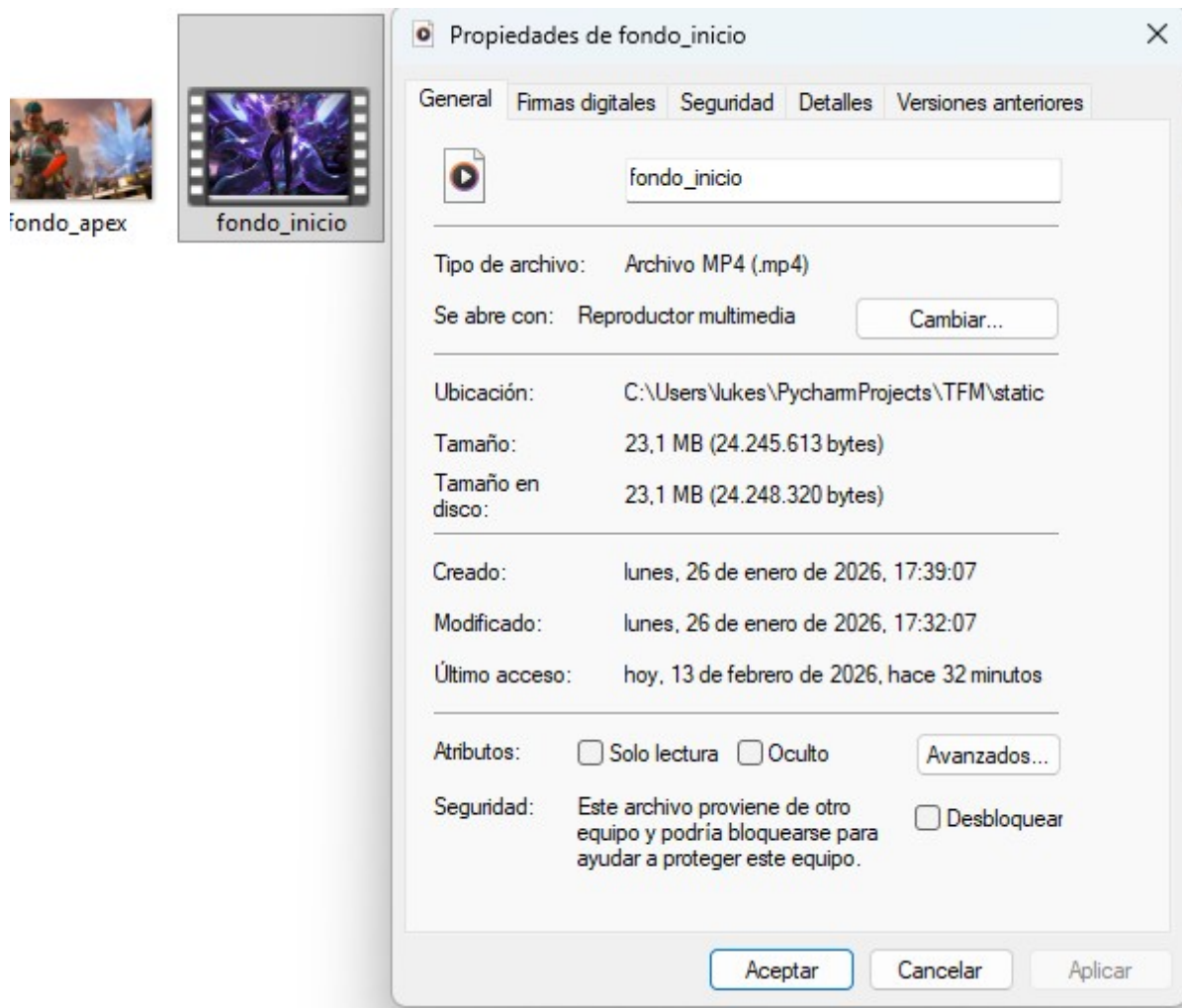


2.3. Limitaciones y Exclusiones

- **Exclusión de Pasarelas de Pago:** No se han incluido pagos reales ni inscripciones con tarjeta de crédito por motivos de seguridad y complejidad técnica fuera del programa.

- **Limitación de Registro de Juegos:** Actualmente el sistema está diseñado para que cada usuario solo pueda tener una misión activa. Si se apunta a un juego nuevo, el sistema reemplaza el anterior. He decidido hacerlo así para simplificar la base de datos y evitar que un usuario "tramposo" se apunte a todo a la vez.

- **Sin Sistema de Chat:** Se ha excluido la comunicación interna entre jugadores. Toda la interacción es del usuario con la plataforma y del administrador con los datos.
- **Optimización de Video:** Debido al peso de los archivos de video (como el fondo de la lista de juegos), la fluidez puede variar según el dispositivo, por lo que he priorizado el funcionamiento del código sobre el peso visual excesivo.





STACK TECNOLÓGICO Y ALTERNATIVAS EVALUADAS

Para construir "All Stars", he seleccionado un conjunto de herramientas que me permitieran trabajar rápido pero con un resultado profesional. He utilizado lenguajes que conozco y tecnologías que me permitieran dar ese acabado futurista a la web.

3.1. Tecnologías de Backend

- **Backend: Python y Flask** He utilizado **Python** como lenguaje principal. He construido la web con **Flask** como framework.

Lo bueno de Flask es que es muy ligero, no te obliga a usar una estructura fija, lo que me ha permitido crear las rutas de los torneos y la lógica de los puntos de una manera muy personalizada.

3.2 Tecnologías de Frontend

Frontend: HTML, CSS3 y Bootstrap 5.

Para la parte visual, Bootstrap ha sido la base. Me ha servido para que la web sea "responsive" y se vea bien en cualquier pantalla. Sin embargo, para que no pareciera una web estándar, he añadido una capa de CSS3 personalizado. Aquí es donde he programado los efectos de cristal, los brillos de neón y las animaciones de los bordes que le dan la estética Cyberpunk.

3.4 Herramientas de Desarrollo y Despliegue



Base de Datos: La base de datos es SQLite ya que es ideal para este proyecto porque no necesita una instalación complicada; los datos se guardan en un archivo .db dentro del proyecto.

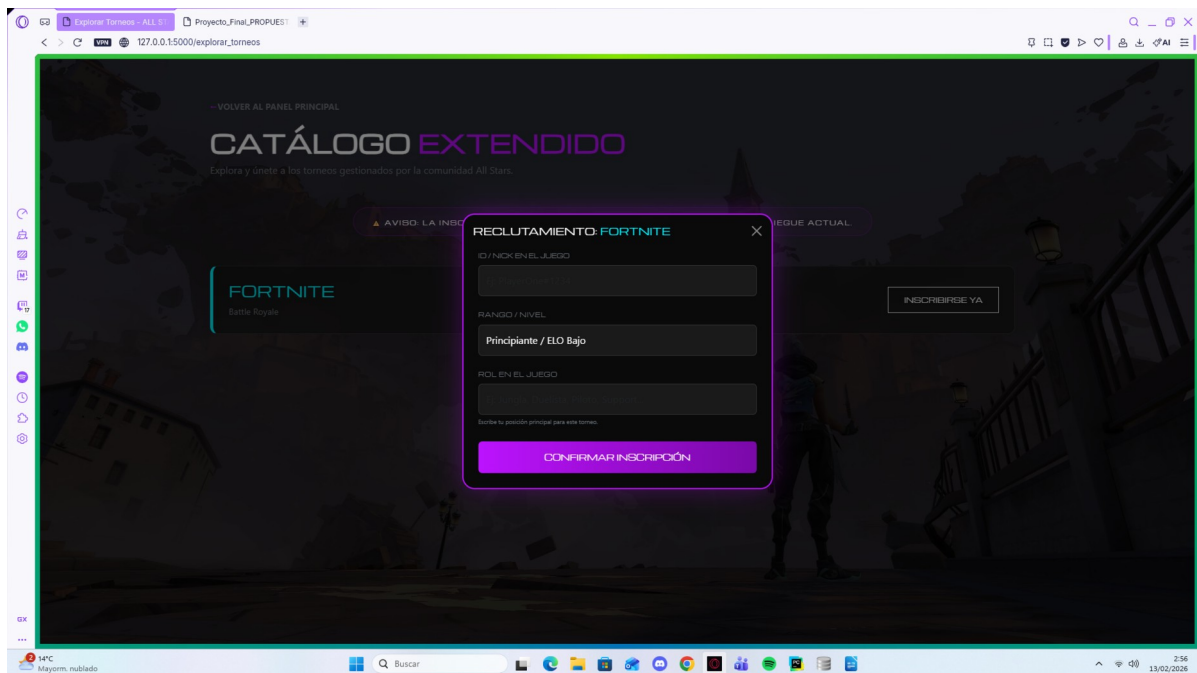
Para comunicarme con ella desde Python, he usado SQLAlchemy, que me permite gestionar a los usuarios y los juegos como si fueran objetos de Python, lo que facilita mucho el trabajo.

Interactividad: JavaScript (Fetch API). He usado JavaScript puro para que la web sea "viva". Lo más importante aquí es la Fetch API, que es la tecnología que permite que el administrador sume puntos a un jugador y el número cambie en pantalla al instante, sin que la página tenga que recargarse entera.

3.2. Alternativas evaluadas

A la hora de empezar consideré otras opciones, como por ejemplo MySQL en lugar de SQLite, pero finalmente me decanté por SQLite pues no requiere de configurar un servidor aparte, además ofrece la misma potencia pero es mucho más fácil de manejar.

Me decanté por Fetch API, simplemente por mejorar la experiencia del usuario y evitarle tener que recargar la página para ver sus puntos actualizados.



MODELO DE DATOS

Para que los puntos, los niveles y las inscripciones de los participantes no se pierda, he diseñado una estructura de datos que permite al administrador tener el control total de lo que ocurre en la plataforma.

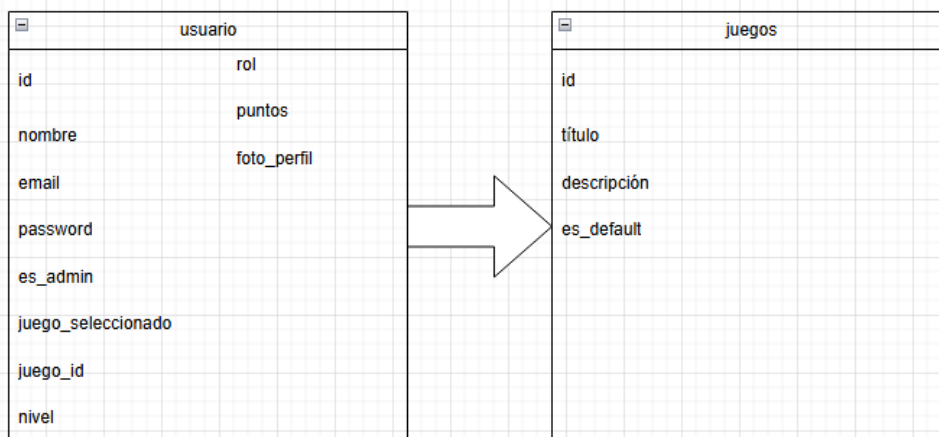
4.1. Arquitectura de la Base de Datos

Para este proyecto he decidido utilizar SQLite como Base de Datos. La arquitectura se basa en un modelo Relacional, por lo que la información se organiza en tablas compuestas por filas y columnas.

Elegí SQLite por su practicidad en un entorno de desarrollo. No requiere de un servidor externo, y toda la base de datos se guarda en un único archivo, con extensión .db, dentro de la carpeta del proyecto.

Para gestionar esta base de datos desde Python, he utilizado SQLAlchemy, que me permite escribir mi código más limpio y seguro.

4.2. Esquema de la Base de Datos





REQUISITOS DE LA APLICACIÓN

5.1. Requisitos Funcionales

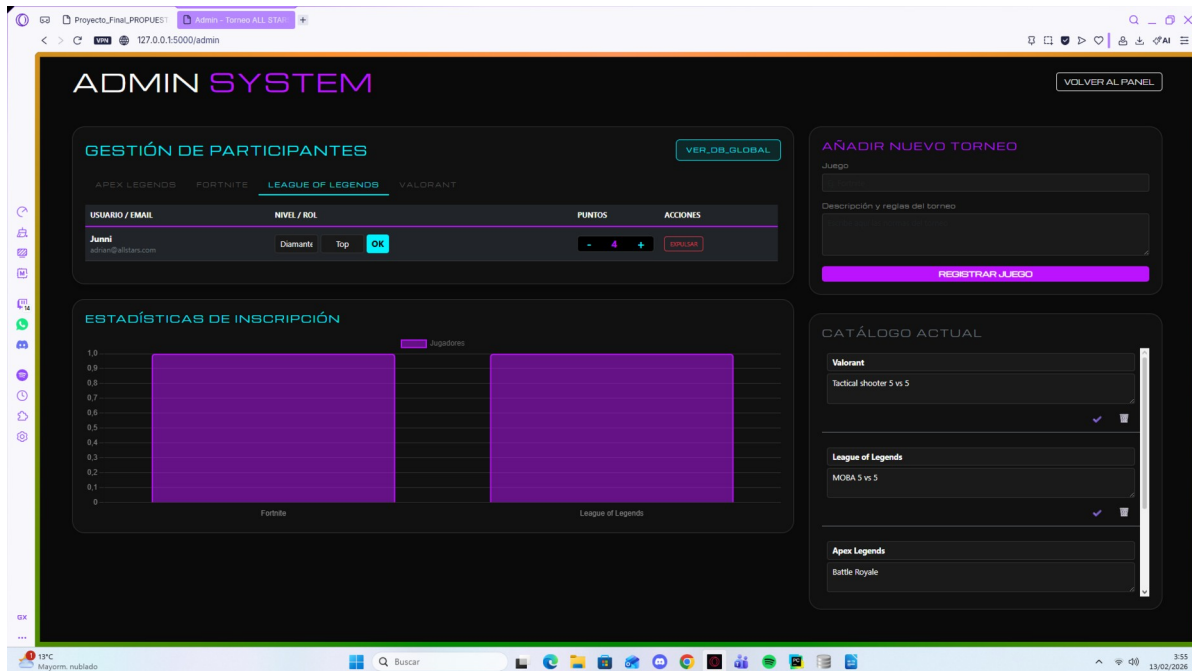
Los requisitos funcionales describen las acciones que el usuario y el administrador pueden realizar dentro de la plataforma.

- **Sistema de Autenticación:** El sistema permite que nuevos usuarios se registren y que los usuarios existentes inicien sesión de forma segura.

- **Gestión de Perfil de Usuario:** El usuario puede visualizar sus estadísticas personales, como sus puntos, el juego al que están inscritos, además de poder editar su foto de perfil, su nombre de usuario y eliminar su cuenta.

- **Inscripción Dinámica en Torneos:** La aplicación presenta un catálogo de juegos extraído de la base de datos. El usuario puede inscribirse en un torneo con un solo clic, actualizando su perfil automáticamente.

- **Panel de Control Administrativo:** El administrador tiene una interfaz exclusiva para gestionar a los participantes. Esto incluye editar niveles, roles y, lo más importante, gestionar la puntuación de los jugadores de forma manual.



- **Visualización Estadística:** El sistema genera gráficas en tiempo real que muestren la distribución de jugadores por cada juego, facilitando la toma de decisiones al administrador, además de ser accesible para los usuarios para poder visualizar el “ranking” del torneo al que están participando.

- **CRUD de Catálogo:** El administrador es capaz de crear, leer, actualizar y borrar (CRUD) los juegos disponibles en el catálogo sin necesidad de tocar el código fuente.

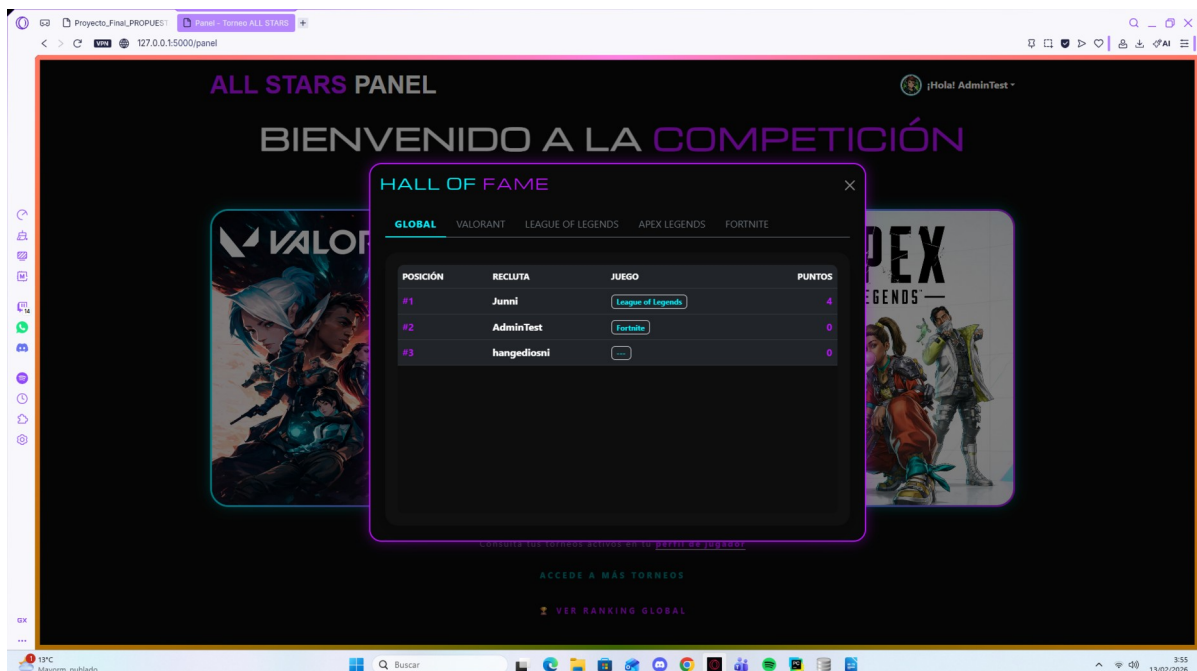
5.2. Requisitos No Funcionales

Los requisitos no funcionales son los que aseguran que la web sea profesional, rápida y agradable de usar.

- **Usuario (UI) Inmersiva:** La web sigue una estética coherente basada en el **Neomorfismo** y el **Glassmorphism**, utilizando colores neón y efectos de transparencia para mejorar la inmersión del usuario.

- **Responsividad:** Gracias al uso de **Bootstrap 5**, la aplicación es totalmente adaptable. Se puede usar tanto en un monitor de PC como en una tablet o smartphone sin que el diseño se rompa.

- **Rendimiento y Actualización Asíncrona:** Mediante el uso de **Fetch API**, las actualizaciones de puntos en el panel de admin se realizan sin recargar la página, garantizando una experiencia fluida y rápida.



- **Portabilidad:** La aplicación es fácil de desplegar. Al utilizar **SQLite**, el sistema no depende de configuraciones complejas de servidores externos para funcionar tras la entrega.

MANUAL DE INSTALACIÓN



Este manual detalla los pasos necesarios para configurar el entorno y ejecutar la aplicación de forma local:

Este proyecto está desarrollado con Python y Flask. Para facilitar su despliegue en cualquier dispositivo, este cuenta con un archivo requirements.txt, el cual requiere de una instalación muy sencilla y rápida.

El proyecto necesitará un intérprete, por tanto, en PyCharm, se debe hacer clic en la parte inferior derecha, donde aparece Python, y seleccionar “Add new interpreter”, y después a “Add local interpreter”

Para instalar todas las dependencias necesarias basta con abrir el terminal en PyCharm y ejecutar “pip install -r requirements.txt”.

La web Torneos ALL STARS ha sido diseñada como una aplicación dinámica que facilita al usuario la inscripción a una variedad de torneos de diferentes videojuegos.

CONCLUSIONES

El desarrollo de Torneos ALL STARS me ha permitido cumplir con mi objetivo principal, crear una plataforma de gestión de torneos que sea, a la vez, funcional y visualmente atractiva.



Para mí, el diseño y la programación deben ir de la mano. No sirve de nada una base de datos perfecta si el usuario no sabe ni cómo inscribirse y de nada sirve una web espectacular si los datos no se guardan correctamente.

Acerca del aprendizaje y la experiencia adquirida, el proyecto ha sido una evolución de ambas en todo momento. Al principio no sabía ni cómo empezar, para mí era impensable implementar Fetch API, pero a medida que iba avanzando, las ideas se me ocurrían solas y me atrevía a indagar y salir de la “zona de comfort”.

Uno de los mayores aprendizajes ha sido sin duda saber buscar soluciones en la documentación técnica, como en los foros de Flask. Aprender a implementar neomorfismo por mi cuenta, saliendo de lo que viene por defecto en Bootstrap, me ha dado mucha más confianza para personalizar futuros proyectos.

Durante el proceso me encontré con retos, como el peso de los archivos multimedia, o errores constantes. Resolver estos problemas me ha enseñado que la programación es, en gran medida, saber descomponer problemas grandes en pasos pequeños y lógicos.

En resumen, la experiencia ha sido muy positiva. He pasado de usar plantillas básicas a construir una aplicación con identidad propia, lo que me motiva a seguir profundizando en el desarrollo de aplicaciones web.



EVOLUTIVOS DEL PROYECTO

Aunque la versión actual de Torneos ALL STARS es totalmente funcional para los objetivos marcados, la plataforma se ha diseñado pensando en una posible mejora, como por ejemplo:

- Sistema de notificaciones en tiempo real para que los usuarios reciban alertas instantáneamente cuando el administrador les sume puntos o anuncie un nuevo torneo.
- Sistema de rangos dinámicos para que se les asigne logros según sus puntos acumulados.
- Historial de actividad que registre cada cambio de puntos, permitiendo al usuario ver un gráfico de su progresión a lo largo del tiempo.
- Tener más de un único torneo activo al mismo tiempo para que los usuarios puedan participar en varios torneos a la vez ya que el sistema actualmente no permite que el usuario se inscriba en dos torneos diferentes ya que la segunda inscripción sobrescribe la primera.
- Implementar un sistema de verificación por email para que el usuario pueda recuperar su contraseña en caso de olvido.